CHAPTER 39

KNN Imputer

K-Nearest Neighbors, Euclidean Distance, nan_euclidean, Weights

CampusX — 100 Days of Machine Learning
Feature Engineering Series, Videos 31–40

39.1 Understanding K-Nearest Neighbors

The **K-Nearest Neighbors (KNN)** algorithm is one of the simplest machine-learning algorithms. To predict a value for a new observation, KNN finds the k most similar existing observations (nearest neighbours) and combines their values (e.g., by averaging for regression, or voting for classification).

KNN imputation applies this idea to missing data: instead of using the global mean or median, it uses the values of the k most similar complete observations to estimate the missing value. This means the imputed value respects the relationships between features.

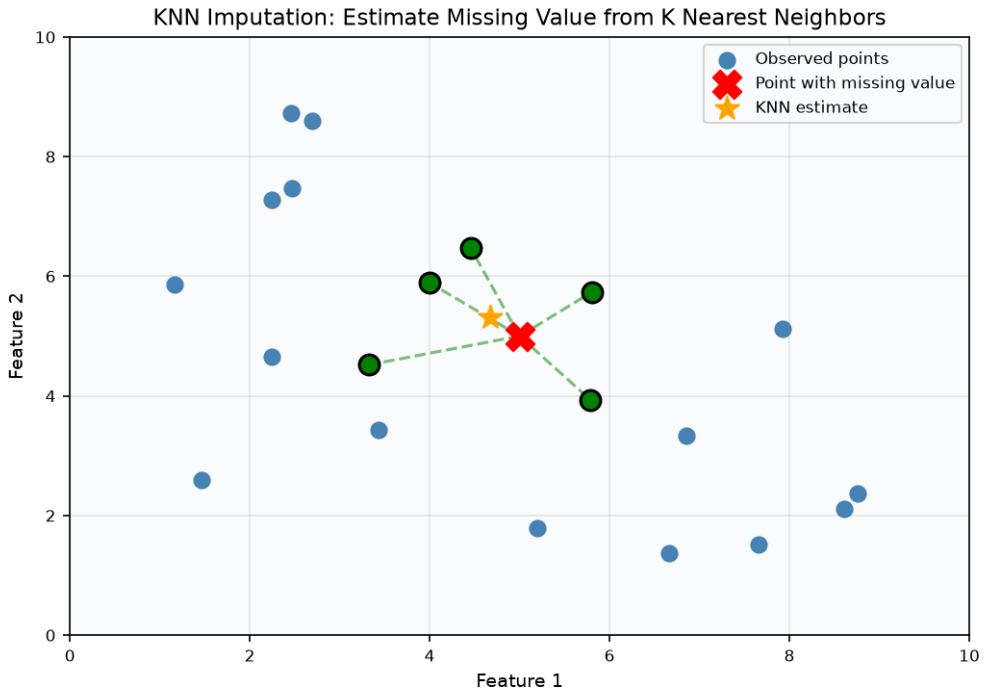


Figure 39.1: KNN imputation concept. The red X marks an observation with a missing value. The 5 nearest neighbours (green circles) are identified, and the missing value is estimated from their values (orange star).

39.2 Mathematical Foundation: Distance

KNN relies on a **distance metric** to determine which observations are "near" each other. The most common metric is **Euclidean distance**:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{p}(x_j - y_j)^2} \tag{39.1}$$

where x and y are two observations with p features each. For two-dimensional points (x■, y■) and (x■, y■), this simplifies to:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{39.2}$$

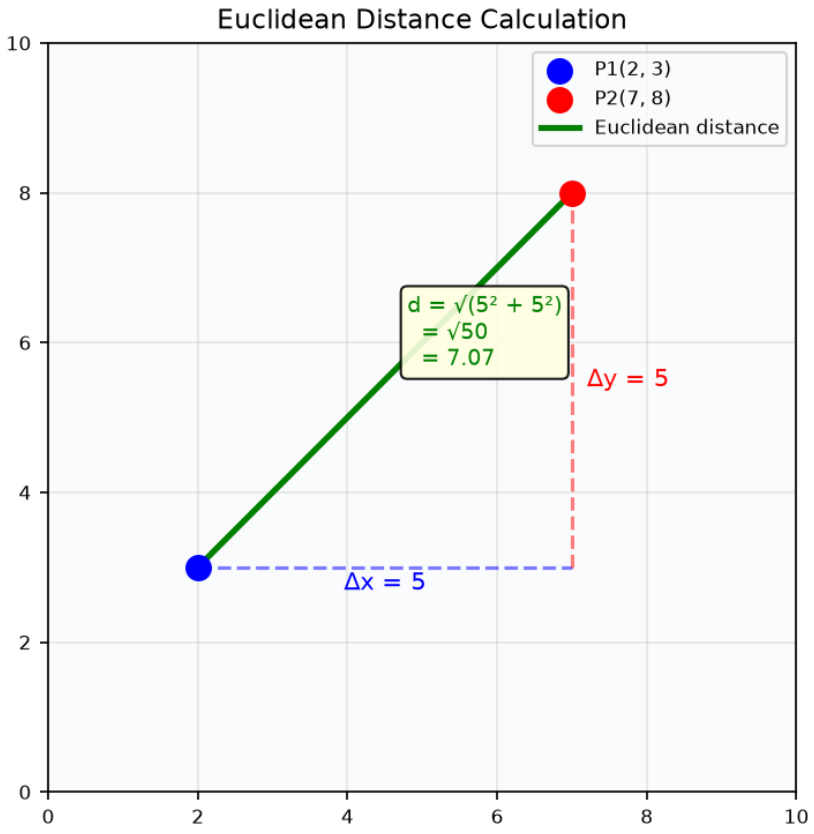


Figure 39.2: Euclidean distance between P1=(2,3) and P2=(7,8). The horizontal and vertical differences form the legs of a right triangle; the distance is the hypotenuse.

39.2.1 Distance with Missing Values

A critical subtlety in KNN imputation: when computing distances between observations, some feature values may be missing. Scikit-learn's KNNImputer handles this by computing the distance using **only the features that are observed in both observations**. Formally, for observations x and y, the distance is computed over the subset of features S where both x■ and y■ are observed:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\frac{p}{|S|}} \cdot \sqrt{\sum_{j \in S}(x_j - y_j)^2} \tag{39.3}$$

The factor p/|S| rescales the distance to account for the missing features, so that distances computed from fewer features are not artificially small.

39.3 KNNImputer in Scikit-learn

Parameter	Default	Description
n_neighbors	5	Number of neighbors to use for imputation
weights	"uniform"	Weight function: "uniform" (equal) or "distance" (inverse distance)
metric	"nan_euclidean"	Distance metric designed for data with NaN values
copy	True	If True, creates a copy; if False, imputes in place

Table 39.1: KNNImputer parameters.

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer

# Create a small example to trace the algorithm manually
data = np.array([
[1.0, 2.0, np.nan],
[2.0, 3.0, 6.0],
[3.0, np.nan, 8.0],
[4.0, 5.0, 10.0],
[5.0, 6.0, np.nan],
[np.nan, 7.0, 12.0]
])

print("Original data:")
print(pd.DataFrame(data, columns=['A', 'B', 'C']))
print()

# Apply KNN imputation with k=2
imputer = KNNImputer(n_neighbors=2, weights='distance')
imputed = imputer.fit_transform(data)

print("After KNN imputation (k=2, weights='distance'):")
print(pd.DataFrame(imputed, columns=['A', 'B', 'C']))
print()

# Trace one imputation manually
# For the missing C in row 0:
# Distance from row 0 to each complete row (using available features)
print("=== MANUAL TRACE: Imputing C[0] (missing) ===")
row0 = data[0] # [1.0, 2.0, NaN]
print(f"Row 0 (with missing): {row0}")

# Compute nan_euclidean distance to each other row
for i in range(1, len(data)):
other = data[i]
# Features observed in both: A and B (C is missing in row 0)
mask = ~np.isnan(row0) & ~np.isnan(other)
n_features_total = data.shape[1]
n_observed = mask.sum()
if n_observed == 0:
print(f" Row {i}: no common features")
continue
sq_diff = np.sum((row0[mask] - other[mask]) ** 2)
distance = np.sqrt(n_features_total / n_observed) * np.sqrt(sq_diff)
print(f" Row {i} {other}: distance = {distance:.4f} (using {n_observed} features)")
```

Listing 39.1: KNNImputer with manual distance trace.

39.4 Weights Parameter

The `weights` parameter controls how neighbouring observations contribute to the imputed value:

Weights	Formula	Behaviour
"uniform"	Each neighbour contributes equally	Imputed value = simple mean of k nearest neighbours
"distance"	Closer neighbours contribute more (inverse distance)	Imputed value = weighted mean, weighted by 1/d

Table 39.2: KNNImputer weights options.

With `weights="distance"`, the imputed value is computed as:

$$\hat{x} = \frac{\sum_{i \in N_k} \frac{x_i}{d_i}}{\sum_{i \in N_k} \frac{1}{d_i}} \tag{39.4}$$

where N■ is the set of k nearest neighbours and d■ is the distance to neighbour i. Closer neighbours (smaller d■) receive higher weight.

39.4.1 Full Pipeline Example

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Realistic dataset
np.random.seed(42)
n = 500
X = pd.DataFrame({
'age': np.random.normal(40, 12, n),
'income': np.random.lognormal(10.5, 0.7, n),
'credit_score': np.random.normal(700, 50, n),
'debt_ratio': np.random.beta(2, 5, n),
'years_employed': np.random.exponential(8, n)
})

# Introduce MAR missingness (age missing more for low income)
for i in range(n):
if X.loc[i, 'income'] < X['income'].quantile(0.3):
if np.random.random() < 0.3:
X.loc[i, 'age'] = np.nan
if np.random.random() < 0.1:
X.loc[i, 'credit_score'] = np.nan
if np.random.random() < 0.15:
X.loc[i, 'debt_ratio'] = np.nan

print("Missing per column:")
print(X.isnull().sum())
print(f"\nTotal missing: {X.isnull().sum().sum()}")

# KNN imputation with different k values
for k in [3, 5, 10]:
imputer = KNNImputer(n_neighbors=k, weights='distance')
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)
print(f"\nk={k}: imputed shape={X_imputed.shape}, "
```

```
f"age mean={X_imputed['age'].mean():.2f}, "
f"credit mean={X_imputed['credit_score'].mean():.1f}")
```

Listing 39.2: KNN imputation pipeline with different k values.

39.5 Advantages, Disadvantages and Computational Cost

39.5.1 Advantages

- **Respects feature relationships:** Unlike mean/median imputation, KNN considers all features when estimating a missing value.
- **No distributional assumption:** Does not assume the data follows any specific distribution.
- **Non-parametric:** Makes no assumptions about the functional form of the data-generating process.
- **Intuitive:** The imputed value is based on actual similar observations.

39.5.2 Disadvantages

- **Computationally expensive:** Requires computing distances between all pairs of observations: $O(n^2 \times p)$ time. For large datasets, this is prohibitive.
- **Memory intensive:** Must store the entire dataset for distance computation.
- **Sensitive to feature scaling:** Features with larger ranges dominate the distance calculation. **Always scale features before KNN imputation.**
- **Sensitive to k:** Too small → noisy estimates; too large → oversmoothing. The optimal k depends on the dataset.
- **Curse of dimensionality:** In high-dimensional spaces, all points become roughly equidistant, reducing the discriminative power of neighbours.

> **Always Scale Before KNN**
> KNN uses distance metrics, so features with larger scales dominate. If one feature ranges from 0–1,000,000 (income) and another from 0–1 (debt ratio), the income feature will completely determine the nearest neighbours. Apply StandardScaler or MinMaxScaler **before** KNN imputation.

39.6 Key Takeaways

> **★ KNNImputer estimates missing values from the k most similar observations using nan_euclidean distance. It respects feature relationships but is computationally expensive and sensitive to feature scaling. Always scale features before applying KNN imputation, and choose k through cross-validation.**

39.7 Interview Questions

1. How does KNNImputer handle missing values when computing distances between observations?
2. Why must you scale features before KNN imputation?
3. What is the computational complexity of KNN imputation?
4. Compare uniform vs distance weights in KNNImputer.
5. When would you choose KNN imputation over mean imputation?